西安交通大学
XI'AN JIAOTONG UNIVERSITY

数电第六次作业

4.1

简单可编程逻辑器件（SPLD）包括可编程只读存储器(PROM)、可编程逻辑阵列器件（PLA）、可编程阵列逻辑器件（PAL），以及通用阵列逻辑器件（GAL）。

它们的共同点是以与阵列和或阵列作为核心逻辑资源，能够以“积之和”的形式实现布尔逻辑函数。

PROM的特点是:与阵列固定，或阵列可编程。

PLA的特点是：与阵列和或阵列都可编程。

PAL的特点是：与阵列可编程，或阵列固定。

GAL的特点是：它是在PAL的基础上，吸收了先进的浮栅技术，输出端都集成着一个可编程的输出逻辑宏单元（OLMC），通用性更强。

4.2 2) 二进制码转至2421

真值表如下：B为二进制码，G为2421码

B3	B2	B1	B0	G10	G3	S2	S1	S0
0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	1
0	0	1	0	0	0	0	1	0
0	0	1	1	0	0	0	1	1
0	1	0	0	0	0	1	0	0
0	1	0	1	0	1	0	1	1
0	1	1	0	0	1	1	0	0
0	1	1	1	0	1	1	0	1
1	0	0	0	0	1	1	1	0
1	0	0	1	0	1	1	1	1
1	0	1	0	1	0	0	0	0
1	0	1	1	1	0	0	0	1
1	1	0	0	1	0	0	1	0
1	1	0	1	1	0	0	1	1
1	1	1	0	1	0	1	0	0
1	1	1	1	1	1	0	0	0

最小项表达式为：

$$G_{10} = \sum(10,11,12,13,14,15)$$

$$G_3 = \sum(5,6,7,8,9,15)$$

$$G_2 = \sum(4,6,7,8,9,14)$$

$$G_1 = \sum(2,3,5,8,9,12,13,15)$$

$$G_0 = \sum(1,3,5,7,9,11,13,15)$$

阵列图为：略

(3)典型Gray码至8421

真值表如下：B为典型Grey码,G为8421码

B3	B2	B1	B0	G10	G3	S2	S1	S0
0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	1
0	0	1	1	0	0	0	1	0
0	0	1	0	0	0	0	1	1
0	1	1	0	0	0	1	0	0
0	1	1	1	0	0	1	0	1
0	1	0	1	0	0	1	1	0
0	1	0	0	0	0	1	1	1
1	1	0	0	0	1	0	0	0
1	1	0	1	0	1	0	0	1
1	1	1	1	1	0	0	0	0
1	1	1	0	1	0	0	0	1
1	0	1	0	1	0	0	1	0
1	0	1	1	1	0	0	1	1
1	0	0	1	1	0	1	0	0
1	0	0	0	1	1	0	0	0

最小项表达式为：

$$G_{10} = \sum(8,9,10,11,14,15)$$

$$G_3 = \sum(12,13)$$

$$G_2 = \sum(4,5,6,7,8,9)$$

$$G_1 = \sum(2,3,4,5,10,11)$$

$$G_0 = \sum(1,2,4,7,8,11,13,14)$$

阵列图为：略

4.3 二位加法器

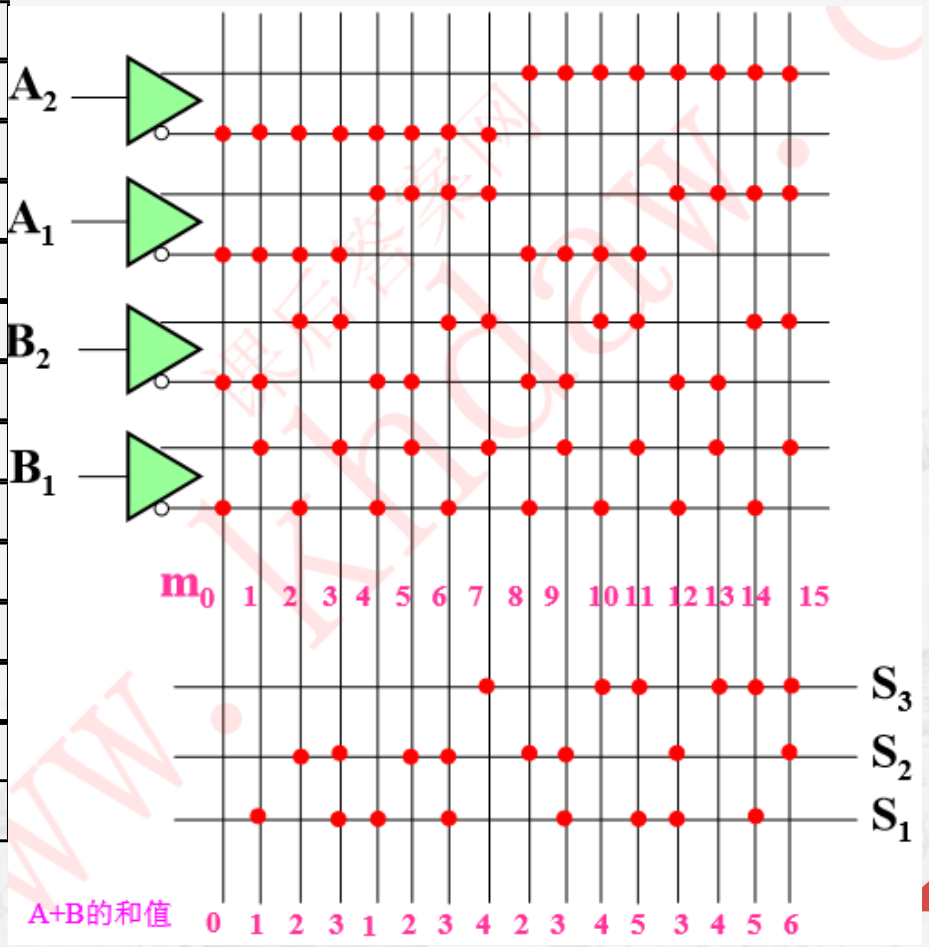
设A,B为两个二位二进制数，S为相加结果，真值表如下：

A2	A1	B2	B1	S3	S2	S1
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	0	0	1	0
0	0	1	1	0	1	1
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	0	1	1
0	1	1	1	1	0	0
1	0	0	0	0	1	0
1	0	0	1	0	1	1
1	0	1	0	1	0	0
1	0	1	1	1	0	1
1	1	0	0	0	1	1
1	1	0	1	1	0	0
1	1	1	0	1	0	1
1	1	1	1	1	1	0

$$S_2 = \sum(1,3,4,6,9,11,12,14)$$

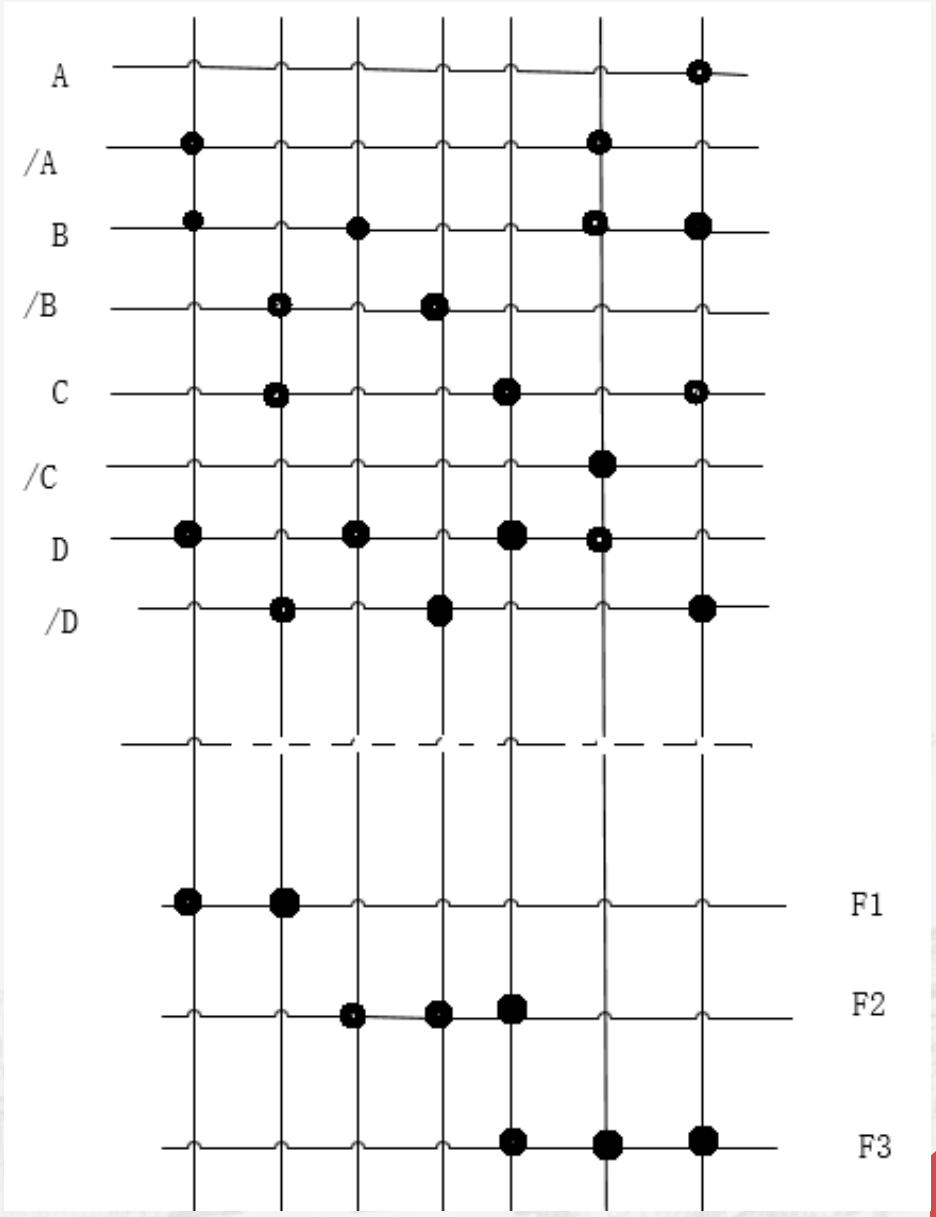
$$S_2 = \sum(2,3,5,6,8,9,12,15)$$

$$S_1 = \sum(7,10,11,13,14,15,)$$



4.4

$$L = \overline{A}B\overline{C}\overline{D} + AB\overline{C}\overline{D} + \overline{B}CD + \overline{A}\overline{B}CD + A\overline{B}\overline{C}\overline{D}$$



4.5

可以利用PLA阵列图表示所要求实现的逻辑函数，如图

4.8列出次态方程

=

=

可参考书**p159**图**4.15**

4.10

GAL器件在PAL器件的基础上做了一些改进。在工艺方面，GAL器件采用高速电可擦除除CMOS的E^2CMOS工艺制作，使其具有了可测试性、低功耗、高集成性、高速率和可重复编程性。在结构方面，GAL器件的结构是通用的，为设计者提供了最大的灵活性。器件内增加了可被编程的保密位，以防对逻辑的复制。

4.13

MAX7000S是在系统可编程的CPLD器件，内部结构是基于乘积项，也就是与或阵列结构。采用E2PROM工艺，采用多电压的I/O接口，其I/O既能够和5V的器件兼容，又能和3.3V的器件兼容。MAX7000是高成本CPLD，规模比较大。

4.15

XC400主要由可编程逻辑块CLB、可配置存储器SRAM阵列、可编程输出输出块以及可编程内部连线PI组成。

CLB给予查找表结构，还包括2个触发器，用来实现时序逻辑。

可配置存储器是一种静态存储器，由两个CMOS反相器和一个用来控制读写的MOS传输开关构成。

可编程输入输出块IOB可以灵活编程，以实现不同逻辑功能，满足不同逻辑的需求。

可编程内部连线PI是XC4000系列中特殊的内部连线，连线资源由水平和垂直的布线通道构成。

4.17

源代码：

```
        LIBRARY  IEEE；
        USE  IEEE.STD_LOGIC_1164.ALL
；


        ENTITY  sop  IS
           PORT (a,b,c,d,e,f:IN
STD_LOGIC；
                    x:OUT STD_LOGIC)  ；
        END sop；

        ARCHITECTURE sop_arc OF sop
IS
           BEGIN
           x<=(a AND b) OR (c AND d) OR
(e AND f)；
        END sop_arc；
```

4.18

源代码：

```
LIBRARY  IEEE；
USE  IEEE.STD_LOGIC_1164.ALL；

ENTITY  boolean  IS
   PORT (a ,b,c :IN STD_LOGIC；
              f :OUT STD_LOGIC)  ；
END boolean ；

ARCHITECTURE boolean_arc OF
boolean IS
    BEGIN
    f< =(a OR (NOT b) OR c) AND (a OR b
OR (NOT c)) AND ((NOT a) OR (NOT b) OR
(NOT c))；
END boolean_arc；
```

4.19

源代码：

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY decoder_3_to_8 IS
  PORT (a,b,c,g1,g2a,g2b:IN STD_LOGIC;
        y:OUT STD_LOGIC_VECTOR(7 downto 0));
END decoder_3_to_8;

ARCHITECTURE rt1 OF decoder_3_to_8 IS
  SIGNAL indata:STD_LOGIC_VECTOR(2 downto 0);
BEGIN
  indata<=c & b & a;
  PROCESS(indata,g1,g2a,g2b)
    BEGIN
      IF (g1='1' AND g2a='0' AND g2b='0')THEN
        CASE indata IS
          WHEN "000"=>y<="11111110";
          WHEN "001"=>y<="11111101";
          WHEN "010"=>y<="11111011";
          WHEN "011"=>y<="11110111";
          WHEN "100"=>y<="11101111";
          WHEN "101"=>y<="11011111";
          WHEN "110"=>y<="10111111";
          WHEN others=>y<="01111111";
        END CASE;
      ELSE
        y<="11111111";
      END IF;
  END PROCESS;
END rt1;
```

4.20

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY gray_count IS
  PORT (clk,y:IN STD_LOGIC;
        qout:OUT STD_LOGIC _VECTOR (2 downto 0));
END gray_count;

ARCHITECTURE arch_gray OF gray_count IS
SIGNAL iq: STD_LOGIC _VECTOR (2 downto 0);
BEGIN
   PROCESS (clk)
     BEGIN
 IF (clk'event AND clk='1') THEN
IF y('1')THEN
   CASE iq IS
              WHEN "000"=>iq<="001";
              WHEN "001"=>iq<="011";
              WHEN "011"=>iq<="010";
              WHEN "010"=>iq<="110";
              WHEN "110"=>iq<="111";
              WHEN "111"=>iq<="101";
              WHEN "101"=>iq<="100";
              WHEN others=>iq<="000";
            END CASE;
         END IF;
```

续

```
IF y('0')THEN
   CASE iq IS
              WHEN "000"=>iq<="100";
              WHEN "100"=>iq<="101";
              WHEN "101"=>iq<="111";
              WHEN "111"=>iq<="110";
              WHEN "110"=>iq<="010";
              WHEN "010"=>iq<="011";
              WHEN "011"=>iq<="001";
              WHEN others=>iq<="000";
            END CASE;
         END IF;
      END IF;
   END PROCESS;
qout<=iq;
END arch_gray;
```



西安交通大学
XI'AN JIAOTONG UNIVERSITY

谢谢大家